

Validação Paralela de Clusters em GPU (Etapa 2 - Modelagem)

Métricas: Índice de Dunn, Coeficiente de Silhueta e Davies-Bouldin

Equipe:

- Henrique M. M. Miranda - 202405479
- Cindy Stephanie Gomes Rabelo - 202403898
- Eduardo Dias Peixoto - 202010395
- Luiany Goncalves Carvalho - 202303351

Disciplina: Computação de Alto Desempenho

Professor: Ricardo Augusto Pereira Franco

UFG - Instituto de Informática

1. O Gargalo de Validação de Clusters

- **O Problema:** Algoritmos de clustering (K-Means, DBSCAN) agrupam os dados, mas validar a qualidade destes agrupamentos exige calcular a proximidade e isolamento dos pontos.
- **O Gargalo:** Métricas como Dunn e Silhueta exigem o cálculo das distâncias euclidianas par-a-par de todos os pontos ($O(N^2)$).
- **Impacto em Big Data:** Para datasets com dezenas de milhares ou milhões de pontos, o cálculo sequencial na CPU torna-se inviável (crescimento quadrático).
- **Métricas Escolhidas:**
 - **Índice de Dunn:** Maximizar separação / dispersão.
 - **Silhueta:** Ajuste local de cada ponto em relação ao cluster vizinho.
 - **Davies-Bouldin:** Razão média entre dispersões e distâncias de centroides.

2. Metodologia de Paralelização em GPU (CUDA)

A. Matriz de Distâncias Euclidiana

- Thread (x, y) calcula a distância $d(x_i, x_j)$. Lançado em grid 2D com blocos de 16×16 threads.

B. Índice de Dunn (Kernel de Redução Paralela)

- Lançamos N blocos de 256 threads (um bloco por ponto/linha da matriz).
- As threads do bloco varrem a linha em paralelo e realizam **Redução em Árvore usando Memória Compartilhada** (`__shared__`) para achar a maior distância intra-cluster e a menor inter-cluster do bloco.
- A CPU apenas faz a redução global linear ($O(N)$) dos resultados dos blocos.

3. Metodologia Paralela: Silhueta e Davies-Bouldin

C. Coeficiente de Silhueta (Shared Memory Dinâmica)

- Um bloco de 256 threads por ponto i .
- **Memória Compartilhada Dinâmica** (`extern __shared__ double shared_sums[]`)
armazena a soma das distâncias de i para cada um dos K clusters (`blockDim.x * K` elementos).
- Cada thread varre os pontos e acumula a distância para o cluster de destino.
- Threads realizam redução local e a thread 0 calcula $s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$.

D. Davies-Bouldin (Operações Atômicas)

- Centroides e dispersões são acumulados em paralelo via `atomicAdd` no Device.
- O cálculo da razão R_{ij} é resolvido em paralelo por cluster.

Validação Numérica Rigorosa

Para garantir a absoluta corretude do cálculo paralelo, validamos as implementações em três camadas de testes independentes:

1. **Caso Analítico Estático para o Índice de Dunn:** Como o `scikit-learn` não implementa Dunn, criamos um caso de teste analítico com 4 pontos conhecidos (Dunn esperado de `2.0`). CPU e GPU obtiveram `2.0` (Erro = $0.00e + 00$).
2. **Comparação com o Scikit-learn (Ground Truth):** Validamos o cálculo de Silhueta e Davies-Bouldin contra o `scikit-learn` oficial para Iris ($N = 150$). Nossos resultados bateram com precisão de máquina (diferença de $\sim 2.28 \times 10^{-9}$).
3. **Equivalência CPU $\equiv$ GPU:** Validamos todas as métricas em todos os tamanhos de datasets do benchmark. Ambas as execuções coincidiram em 100% dos testes.

4. Experimentos e Resultados (CPU vs GPU)

Os códigos foram testados sobre dados sintéticos (blobs) de dimensão $D = 4$ e $K = 5$ clusters. Os tempos coletados comparam a CPU sequencial com a GPU paralela (NVIDIA T4 no Colab):

Pontos (N)	Tempo CPU (s)	Tempo GPU (s)	Speed-up (x)	Corretude
250	0.0010	0.0012	0.87x	100% Match
500	0.0032	0.0016	1.97x	100% Match
1000	0.0113	0.0026	4.29x	100% Match
2000	0.0456	0.0051	8.88x	100% Match
4000	0.2187	0.0147	14.89x	100% Match
8000	0.9191	0.0274	33.51x	100% Match

Atingimos um Speed-up máximo de 33.51x com 8.000 pontos. O gráfico com as curvas foi

5. Pipeline de Execução no Google Colab (GPU NVIDIA)

Para rodar os kernels CUDA na GPU de forma real (contornando a falta de GPU NVIDIA local), estruturamos um pipeline automático:

1. **Notebook Colab:** Configurado com acelerador **T4 GPU** ativo.
2. **Arquivos:** Faz-se o upload de `baseline_cpu.cpp`, `metrics_cuda.cu` e `benchmark.py`.
3. **Automação:** O script `benchmark.py` é disparado:
 - Compila a CPU com `g++` e a GPU com `nvcc`.
 - Executa os testes de escalabilidade, valida a precisão matemática das métricas (corretude 100% CPU vs GPU).
 - Mede os tempos reais de processamento e exporta o gráfico consolidado de **Speed-up** (`curva_performance_cuda.png`).

6. Próximos Passos (Etapa Final)

Para a consolidação da etapa de implementação final do projeto, planejamos:

1. **Otimizações no Kernel de Distância:** Implementar *Tiling* com memória compartilhada para minimizar a banda de memória global.
2. **Uso de Streams no CUDA:** Executar transferências de dados de forma assíncrona concorrentemente com o processamento dos Kernels (overlap de H2D/D2H).
3. **Testes de Larga Escala:** Avaliar o limite de memória da GPU e barramento PCIe escalando o dataset para $N = 50.000$ e $N = 100.000$ registros.
4. **Relatório Final:** Construção da monografia final descrevendo a engenharia dos kernels e análise comparativa detalhada de speed-up.

Obrigado!

Abertos a dúvidas e sugestões.